Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Новгородский государственный университет имени Ярослава Мудрого»
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ

РАЗРАБОТКА БРАУЗЕРНОЙ ОНЛАЙН-ИГРЫ ALIAS

Курсовой проект

ПМ.05 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННЫХ СИСТЕМ

Пояснительная записка курсового проекта по специальности

09.02.07 Информационные системы и программирование

Студентка группы 3993

_____/ В.О. Владимирова /

«___»_____2026 г.

Оценка _____

Руководитель

_____/ В.Р. Ильин /

«___»_____2026 г.

Аннотация

В данной работе разработана браузерная онлайн-игра «Alias», реализующая многопользовательскую версию популярной словесной игры в режиме реального времени. Целью проекта является создание системы, позволяющей пользователям взаимодействовать независимо от их географического положения.

Приложение включает функциональность регистрации и авторизации пользователей, создания игровых комнат, голосового общения и ведения статистики. В качестве технологического стека использованы React, Node.js, PostgreSQL, Redis, WebSocket и WebRTC.

В ходе работы были реализованы основные программные модули, проведено функциональное и нагрузочное тестирование, а также выполнен деплой системы в production-среду. Результаты тестирования подтвердили корректность работы приложения и соответствие установленным требованиям.

Руководство пользователя представлено в Приложении Д.

Глоссарий

API (Application Programming Interface, программный интерфейс приложения) — совокупность правил и протоколов, определяющих способы взаимодействия программных компонентов между собой.

CDN (Content Delivery Network, сеть доставки контента) — распределённая сеть серверов для ускоренной доставки контента пользователям за счёт географической близости к ним.

CI/CD (Continuous Integration / Continuous Delivery, непрерывная интеграция и доставка) — практика автоматического запуска сборки, тестирования и развёртывания приложения при каждом изменении исходного кода.

CORS (Cross-Origin Resource Sharing, совместное использование ресурсов между источниками) — механизм браузера, управляющий доступом веб-страницы к ресурсам стороннего домена.

DNS (Domain Name System, система доменных имён) — система преобразования доменных имён в IP-адреса.

Docker — платформа контейнеризации, обеспечивающая упаковку приложений с зависимостями в изолированные контейнеры для воспроизводимого развёртывания.

Docker Compose — инструмент запуска многоконтейнерных приложений на основе единого конфигурационного файла.

ESLint — статический анализатор кода для JavaScript и TypeScript, обеспечивающий выявление ошибок и единый стиль кодирования.

Express — минималистичный HTTP-фреймворк для Node.js с поддержкой маршрутизации и промежуточного программного обеспечения.

GitHub Actions — встроенный в GitHub сервис для автоматизации сборки, тестирования и развёртывания проектов.

HMR (Hot Module Replacement, горячая замена модулей) — механизм обновления изменённых модулей приложения в браузере без полной перезагрузки страницы.

Jest — фреймворк для написания и выполнения автоматических тестов JavaScript/TypeScript-кода с поддержкой измерения покрытия.

JWT (JSON Web Token) — компактный токен с цифровой подписью для хранения и передачи данных пользовательской сессии.

k6 — инструмент нагрузочного тестирования, имитирующий одновременную работу множества виртуальных пользователей.

localStorage — браузерное хранилище пар «ключ — значение» на стороне клиента без ограничения срока хранения.

NAT (Network Address Translation, трансляция сетевых адресов) — механизм выхода нескольких устройств локальной сети в Интернет через один публичный IP-адрес.

Node.js — среда выполнения JavaScript на стороне сервера с асинхронной событийно-ориентированной моделью.

ORM (Object-Relational Mapping, объектно-реляционное отображение) — технология работы с реляционной базой данных посредством объектов языка программирования.

PostgreSQL — объектно-реляционная система управления базами данных с открытым исходным кодом.

Prisma ORM — инструмент типобезопасного доступа к базам данных для Node.js/TypeScript с поддержкой миграций схемы.

React — JavaScript-библиотека построения пользовательских интерфейсов на основе компонентов с виртуальным DOM.

Redis — высокопроизводительное хранилище данных в оперативной памяти, применяемое как кэш.

REST API (Representational State Transfer) — архитектурный стиль взаимодействия клиента и сервера поверх HTTP на основе операций с ресурсами.

scrypt — криптографическая функция формирования ключа из пароля, устойчивая к атакам за счёт высоких требований к памяти.

SPA (Single Page Application, одностраничное веб-приложение) — веб-приложение, в котором навигация и обновление данных выполняются без перезагрузки страницы.

SSL (Secure Sockets Layer) — протокол шифрования данных при передаче между клиентом и сервером.

STUN-сервер (Session Traversal Utilities for NAT) — сервер, позволяющий клиентам за NAT обнаруживать свой публичный адрес для установки прямого соединения.

Supertest — библиотека тестирования HTTP-маршрутов Express-приложения без запуска реального сервера.

TURN-сервер (Traversal Using Relays around NAT) — ретрансляционный сервер для передачи медиапотока при невозможности прямого WebRTC-соединения.

TypeScript — надмножество JavaScript со статической типизацией, транспилируемое в JavaScript перед выполнением.

VPS (Virtual Private Server, виртуальный выделенный сервер) — виртуальная машина с выделёнными ресурсами, предоставляемая хостинг-провайдером.

Vite — инструмент сборки фронтенд-приложений с быстрым стартом на основе нативных ES-модулей и поддержкой HMR.

WebRTC (Web Real-Time Communication) — браузерная технология передачи аудио, видео и данных между пользователями напрямую (peer-to-peer).

WebSocket — протокол постоянного двустороннего соединения между клиентом и сервером для обмена данными в реальном времени.

XSS (Cross-Site Scripting, межсайтовый скриптинг) — уязвимость, при которой злоумышленник внедряет вредоносный скрипт в страницы, просматриваемые другими пользователями.

Содержание

Введение 7
1 Аналитико-проектная часть 9
1.1 Постановка задачи и требования к системе 9
1.2 Проектирование информационной системы 15
1.3 Инструментальные средства и организация разработки 19
2 Реализация и тестирование 23
2.1 Реализация программных компонентов 23
2.2 Тестирование и оценка качества 27
Заключение 31
Список использованных ресурсов 32
Приложение А 33
Приложение Б 37
Приложение В 40
Приложение Г 48
Приложение Д 55

Введение

В условиях стремительного развития информационных технологий всё большую популярность приобретают веб-приложения, обеспечивающие взаимодействие пользователей в режиме реального времени. Особый интерес представляют игровые платформы, позволяющие объединять пользователей независимо от их географического положения.

Настольная игра «Alias» широко используется для развития коммуникативных навыков, однако её классическая версия имеет ряд ограничений. Основными из них являются необходимость физического присутствия участников, ограниченный набор слов и отсутствие возможности персонализации игрового процесса.

В связи с этим разработка веб-приложения для реализации онлайн-версии игры «Alias» является актуальной задачей. Такое решение позволяет обеспечить доступ к игре из любой точки мира, расширить функциональные возможности, а также повысить удобство и интерактивность игрового процесса.

Целью работы является разработка веб-приложения для реализации онлайн-версии игры «Alias» с поддержкой многопользовательского режима и расширенных функциональных возможностей.

Для достижения поставленной цели необходимо решить следующие задачи:

— провести анализ предметной области;

— сформулировать требования к системе;

— спроектировать архитектуру приложения;

— реализовать программные компоненты;

— выполнить тестирование и оценку качества системы.

Объект исследования — веб-приложения реального времени.

Предмет исследования — процессы разработки и реализации многопользовательской игровой системы.

Работа состоит из двух разделов. В первом разделе выполнен анализ предметной области, сформулированы требования и проведено проектирование системы. Второй раздел посвящён реализации программных компонентов и результатам тестирования.

1 Аналитико-проектная часть

1.1 Постановка задачи и требования к системе

1.1.1 Обоснование необходимости разработки

Настольная игра «Alias» — одна из наиболее популярных словесных игр, в которой участники объясняют слова без использования однокоренных форм. Классическая версия игры ограничена физическим присутствием участников и комплектом карточек, что существенно сужает аудиторию и возможности.

Разработка веб-версии решает следующие проблемы:

— географическая ограниченность: участники не могут играть удалённо;

— ограниченный набор слов: физический набор содержит фиксированное количество карточек, слова быстро запоминаются;

— отсутствие персонализации: нет возможности добавить слова на специфическую тематику;

— отсутствие системы хранения статистики: результаты игр нигде не фиксируются и не накапливаются.

В связи с этим разработка веб-приложения предоставляет следующие преимущества для пользователей:

— доступ к игре из любой точки мира;

— создание собственных тематических коллекций слов;

— наличие голосовой связи в рамках игровой комнаты;

— ведение личной статистики (победы, поражения, угаданные и пропущенные слова);

— добавление пользователей в друзья.

1.1.2 Описание предметной области

В системе выделяются следующие участники:

— игрок: пользователь, участвующий в игровой сессии;

— хост: пользователь, создающий игровую комнату и управляющий её настройками;

— гость: пользователь, подключающийся к существующей игровой комнате по коду или ссылке.

В системе выделяются следующие объекты:

— комната: игровая сессия с уникальным идентификатором, настройками (таймер, победный счёт, сложность, выбор коллекций) и списком игроков;

— коллекция слов: именованный набор карточек; бывает стандартной (системной) или пользовательской;

— карточка: слово с уровнем сложности (1–3);

— игровой раунд: ход одного игрока, в течение которого активный игрок объясняет слова, а остальные участники угадывают в чате.

В рамках системы реализованы следующие бизнес-правила:

— создание комнаты доступно как авторизованным, так и гостевым пользователям;

— игру может запустить только хост;

— запуск игры возможен при наличии не менее двух участников;

— начисление очков осуществляется следующим образом: за угаданное слово начисляется +1 очко активному и угадавшему игроку, за пропуск снимается −1;

— победа фиксируется при достижении настроенного количества очков;

— пользователь может находиться только в одной комнате одновременно;

— верификация электронной почты не является обязательным условием для участия в игре, однако рекомендуется для предотвращения потери доступа к аккаунту.

1.1.3 Характеристика процессов функционирования

В Приложении А приведены диаграммы основных бизнес-процессов системы (рисунки А.1–А.2).

Границы системы и взаимодействие с внешними сущностями представлены на рисунке А.3.

Система предусматривает следующие основные сценарии взаимодействия пользователя:

— регистрация и вход в систему: пользователь регистрируется, указывая имя пользователя, электронную почту и пароль, после валидации данных сервер создаёт учётную запись и отправляет письмо подтверждения, при входе пользователь вводит имя и пароль, которые проверяются сервером, в случае успешной аутентификации создаётся сессия;

— создание и участие в игре: хост создаёт игровую комнату, после чего приглашает других пользователей, игроки подключаются к комнате и ожидают начала игры, после запуска начинается игровой процесс с поочерёдными ходами участников;

— игровой процесс: во время хода активному игроку передаётся слово, которое необходимо объяснить, остальные участники пытаются угадать слово через чат, система отслеживает правильные ответы, начисляет очки и управляет сменой ходов;

— голосовое взаимодействие: пользователи могут общаться голосом в рамках игровой комнаты; соединение устанавливается между клиентами с использованием WebRTC и публичного STUN-сервера для прохождения NAT; применение TURN-сервера не предусмотрено, что является ограничением в сетевых средах с ограничительными политиками;

— работа с коллекциями слов: пользователь может создавать, редактировать и использовать собственные наборы слов, при запуске игры выбранные коллекции формируют общий пул слов.

Последовательности операций в системе включают следующие процессы:

— регистрация пользователя: ввод регистрационных данных, отправка данных на сервер, валидация данных, создание записи в базе данных, отправка письма подтверждения;

— аутентификация: ввод логина и пароля, передача данных серверу, проверка хэша пароля, создание пользовательской сессии;

— создание игровой комнаты: отправка запроса на создание комнаты, генерация уникального идентификатора, сохранение состояния комнаты, возврат данных клиенту;

— подключение к комнате: получение ссылки или кода комнаты, установление WebSocket-соединения, регистрация игрока в комнате;

— запуск игры: инициирование запуска хостом, проверка готовности игроков, запуск таймера и игрового цикла;

— игровой цикл: выбор слова, передача слова активному игроку, приём ответов от других игроков, проверка правильности, начисление очков, завершение хода и передача следующему игроку;

— завершение игры: проверка достижения очков для победы, определение победителя, сохранение результатов.

1.1.4 Функциональные и нефункциональные требования

1.1.4.1 Функциональные требования

Система должна обеспечивать выполнение следующих функций:

— регистрация пользователей с проверкой корректности введённых данных (имя пользователя, пароль, электронная почта);

— аутентификация пользователей по имени пользователя и паролю;

— подтверждение электронной почты пользователя посредством одноразовой ссылки;

— восстановление доступа к учётной записи через электронную почту;

— создание и присоединение пользователей к игровой комнате;

— настройка параметров игровой комнаты (время хода, победный счёт, уровень сложности, выбор коллекций слов);

— обеспечение взаимодействия пользователей в режиме реального времени в рамках игровой сессии;

— обмен текстовыми сообщениями между участниками игровой сессии;

— обеспечение голосового взаимодействия между участниками игровой комнаты;

— создание, редактирование и удаление пользовательских коллекций слов;

— предоставление возможности выбора стандартных коллекций слов при создании игровой комнаты;

— управление списком друзей (поиск пользователей, отправка и обработка заявок в друзья);

— предоставление статистики пользователя (количество побед, поражений, успешных и пропущенных ответов);

— редактирование профиля пользователя (изменение имени пользователя и аватара);

— сохранение результатов завершённых игровых сессий.

1.1.4.2 Нефункциональные требования

Требования к производительности:

— время отклика системы (p95) не превышает 500 мс при нагрузке до 20 виртуальных пользователей;

— задержка передачи данных в режиме реального времени не превышает 100 мс.

Требования к безопасности:

— обеспечивается хэширование паролей пользователей с использованием алгоритма scrypt;

— обеспечивается защита от межсайтового скриптинга (XSS) средствами экранирования данных на стороне клиентского приложения (React Escaping);

— выполняется валидация всех входных данных, поступающих в систему;

— обеспечивается поддержка механизма CORS для управления междоменным доступом.

Требования к надёжности:

— обеспечивается автоматический переотбор слов при исчерпании доступного пула игровых слов;

— обеспечивается корректное завершение игровой комнаты при отключении участников.

Требования к масштабируемости:

— обеспечивается использование механизма кэширования коллекций данных (Redis);

— предусмотрена возможность контейнеризации компонентов системы (Docker) для масштабирования и развёртывания.

Требования к совместимости:

— обеспечивается поддержка современных веб-браузеров;

— обеспечивается частичная адаптация пользовательского интерфейса под различные устройства и разрешения экрана.

Требования к доступности:

— обеспечивается поддержка локализации пользовательского интерфейса;

— сообщения об ошибках представляются в понятной для пользователя форме.

1.2 Проектирование информационной системы

1.2.1 Выбор архитектурных решений

Для разработки многопользовательской игровой системы «Alias» выбрана клиент-серверная архитектура, реализованная в виде двух независимых компонентов: одностраничного веб-приложения (SPA), функционирующего на стороне клиента, и REST API-сервера с поддержкой WebSocket на стороне сервера.

Выбор данной архитектуры обусловлен требованиями обеспечения взаимодействия пользователей в режиме реального времени. Игровой процесс предполагает необходимость мгновенной синхронизации состояния между участниками, включая передачу текущего слова, таймера и результатов игры. Для реализации указанных требований используется технология WebSocket, обеспечивающая постоянное двустороннее соединение между клиентом и сервером без необходимости выполнения повторных HTTP-запросов.

Применение клиент-серверной архитектуры позволяет обеспечить чёткое разделение ответственности между компонентами системы. Клиентская часть отвечает за отображение пользовательского интерфейса и обработку действий пользователя, в то время как серверная часть реализует всю бизнес-логику системы. Данный подход повышает уровень безопасности, поскольку исключает возможность изменения игровой логики на стороне клиента.

Архитектурное решение также обеспечивает высокую масштабируемость системы. Разделение компонентов позволяет выполнять независимое развёртывание и масштабирование клиентской и серверной частей. Для контейнеризации используется технология Docker, что упрощает процесс развёртывания и сопровождения системы.

В качестве альтернативных архитектурных решений рассматривались монолитная и микросервисная архитектуры. Монолитная архитектура была отклонена в связи с ограниченной масштабируемостью, микросервисная —

ввиду избыточной сложности реализации и сопровождения для данного уровня проекта.

1.2.2 Структура и состав компонентов

Разработанная информационная система состоит из следующих основных компонентов: клиентского приложения, серверной части, базы данных, системы кэширования и файлового хранилища.

Клиентское приложение реализовано с использованием библиотеки React и языка программирования TypeScript и представляет собой одностраничное приложение (SPA). Основными функциями клиентской части являются отображение пользовательского интерфейса, управление состоянием приложения, обработка пользовательских действий, а также взаимодействие с серверной частью посредством REST API и WebSocket. Дополнительно реализована поддержка голосового общения на основе технологии WebRTC, при этом сигнализация осуществляется через WebSocket. Для хранения данных сессии используется JWT-токен, сохраняемый в localStorage. Защита от XSS-атак обеспечивается механизмом автоматического экранирования данных библиотеки React, что предотвращает внедрение вредоносных скриптов в контексте приложения.

Серверная часть системы реализована на платформе Node.js с использованием фреймворка Express и языка TypeScript. Сервер обеспечивает обработку HTTP-запросов, управление WebSocket-соединениями, реализацию игровой логики, а также выполнение бизнес-операций, связанных с пользователями, игровыми комнатами, коллекциями слов и социальной составляющей системы.

В качестве системы управления базами данных используется PostgreSQL. Взаимодействие с базой данных осуществляется с использованием ORM Prisma, обеспечивающего типобезопасность запросов и управление миграциями структуры данных.

Для повышения производительности системы применяется кэширование на основе Redis. В кэш предварительно загружаются

стандартные коллекции слов, что позволяет снизить нагрузку на базу данных и ускорить получение данных в процессе игры.

Файловое хранилище используется для хранения пользовательских аватаров. Файлы размещаются на сервере в локальной директории, доступ к ним осуществляется посредством отдельного HTTP-маршрута.

Взаимодействие между компонентами системы осуществляется по следующим протоколам: клиент взаимодействует с сервером посредством HTTP и WebSocket, сервер взаимодействует с базой данных PostgreSQL и Redis по протоколу TCP, а также с внешним почтовым сервисом через HTTPS.

Архитектура системы на уровне контейнеров представлена на рисунке А.4.

1.2.3 Проектирование программных модулей

В Приложении А представлена компонентная структура backend-сервиса (рисунок А.5) и frontend-приложения (рисунок А.6).

Серверная часть системы реализована в виде набора логически выделенных программных модулей, каждый из которых выполняет строго определённую функциональную задачу.

Модули маршрутизации обеспечивают обработку входящих HTTP-запросов и включают функциональность регистрации и авторизации пользователей, управления профилем, игровыми комнатами, списком друзей и коллекциями слов.

Модули бизнес-логики реализуют основную функциональность системы. Модуль аутентификации отвечает за регистрацию, авторизацию и управление учетными записями пользователей. Игровой модуль реализует логику игрового процесса, включая управление игровым циклом, выбор слов, обработку действий игроков и формирование результатов игры. Модуль работы с друзьями обеспечивает обработку заявок и формирование списка контактов пользователей. Модуль коллекций слов отвечает за создание, хранение и получение наборов слов. Дополнительно реализованы сервисы

кэширования данных, отправки электронных писем и управления состоянием пользователей в сети.

Отдельно выделен модуль WebSocket, который обеспечивает обработку постоянных соединений, синхронизацию состояния игровых комнат между участниками, передачу игровых событий, а также поддержку голосового общения посредством WebRTC.

Клиентская часть системы также структурирована на набор модулей, включающих сервисы взаимодействия с API, управление состоянием аутентификации, обработку сессий пользователя, реализацию голосового чата и модуль интернационализации интерфейса.

1.2.4 Проектирование структуры данных

Структура базы данных разработана на основе реляционной модели и включает набор взаимосвязанных таблиц, обеспечивающих хранение данных информационной системы. Модель базы данных приведена в Приложении А на рисунке А.7.

Основной сущностью является таблица пользователей, предназначенная для хранения учётных записей. Таблица содержит идентификатор пользователя, имя пользователя, адрес электронной почты, хэш пароля, дату регистрации и статус подтверждения электронной почты. Данная таблица связана с таблицей статистики пользователей отношением «один к одному», в которой хранится информация об игровых показателях, включая количество побед, поражений, угаданных и пропущенных слов.

Социальная подсистема реализована с использованием таблицы отношений дружбы между пользователями и таблицы заявок в друзья, обеспечивающих установление связей между пользователями по принципу «многие ко многим».

Информация об игровых сессиях хранится в таблице игр, содержащей сведения о времени начала и окончания игры, составе участников, итоговом результате и победителе.

Система словарей реализована с помощью таблиц стандартных и пользовательских коллекций слов, включающих название, описание, уровень сложности и количество элементов.

Дополнительно предусмотрена таблица токенов электронной почты, предназначенная для подтверждения регистрации пользователей и восстановления пароля. Каждый токен является уникальным и имеет ограниченный срок действия.

Таким образом, разработанная структура базы данных обеспечивает хранение всей необходимой информации для функционирования информационной системы.

1.3 Инструментальные средства и организация разработки

1.3.1 Выбор среды разработки и вспомогательных инструментов

В процессе разработки информационной системы использовался набор современных инструментальных средств, обеспечивающих эффективную разработку, тестирование и развёртывание программного продукта.

Основной средой разработки является Visual Studio Code, выбранная благодаря широкой поддержке языка TypeScript, фреймворка React и большого количества расширений, упрощающих разработку и отладку приложений.

Контейнеризация компонентов системы осуществляется с использованием Docker и Docker Compose, что обеспечивает воспроизводимость среды разработки, изоляцию сервисов и упрощение процесса развёртывания.

Тестирование системы осуществляется с использованием Jest для модульного и интеграционного тестирования, Supertest для тестирования HTTP-эндпоинтов Express-приложения, а также k6 для проведения нагрузочного тестирования и анализа производительности системы.

Для отправки электронных писем используется сервис Resend, предоставляющий стабильный API для транзакционных сообщений. Контроль качества кода осуществляется с использованием ESLint, обеспечивающего единый стиль кодирования и выявление потенциальных ошибок на этапе разработки.

Процессы непрерывной интеграции и доставки (CI/CD) реализованы с использованием GitHub Actions, обеспечивающих автоматический запуск тестов и сборку проекта при внесении изменений в основную ветку.

Развёртывание серверной части осуществляется на VPS-платформе Timeweb Cloud с использованием Docker. Управление доменом, SSL-сертификатами и защитой трафика реализовано через сервис Cloudflare, который также выполняет функции CDN и DNS-провайдера.

1.3.2 Используемые технологии и библиотеки

1.3.2.1 Клиентская часть

Разработка клиентской части выполнена с использованием библиотеки React версии 19, обеспечивающей компонентный подход к построению пользовательского интерфейса и высокую производительность за счёт виртуального DOM.

Для типизации кода используется TypeScript, что позволяет снизить количество ошибок на этапе выполнения и повысить надёжность приложения.

В качестве инструмента сборки используется Vite, обеспечивающий высокую скорость разработки за счёт использования нативных ES-модулей и механизма горячей перезагрузки (HMR).

Для реализации взаимодействия в реальном времени используется встроенный WebSocket API, обеспечивающий обмен данными между клиентом и сервером.

Дополнительно применяется WebRTC API для реализации голосового чата между пользователями; для прохождения NAT применяется публичный STUN-сервер. Использование TURN-сервера не предусмотрено,

что является ограничением системы в сетевых средах с ограничительными политиками маршрутизации.

1.3.2.2 Серверная часть

Серверная часть реализована на платформе Node.js версии 18 и выше, что позволяет использовать единый язык программирования (TypeScript) на клиенте и сервере и обеспечивает асинхронную обработку запросов.

В качестве HTTP-фреймворка используется Express версии 5, обеспечивающий гибкость маршрутизации и простоту настройки серверной логики.

Для работы с базой данных применяется Prisma ORM, обеспечивающий типобезопасные запросы, автоматическую генерацию клиента и поддержку миграций.

Обработка WebSocket-соединений реализована с использованием библиотеки ws, отличающейся высокой производительностью и минимальными накладными расходами.

Для взаимодействия с Redis используется официальный клиент redis, поддерживающий асинхронную модель программирования.

Для реализации системы аутентификации и защиты данных применяется встроенный криптографический модуль Node.js (crypto) с использованием алгоритма scrypt.

Отправка электронных сообщений реализована через сервис Resend, обеспечивающий надёжную доставку транзакционных email.

Для управления конфигурацией приложения используется библиотека dotenv, позволяющая хранить переменные окружения вне исходного кода.

1.3.3 Организация разработки с использованием контроля версий (Git)

В процессе разработки информационной системы использовалась система контроля версий Git. Исходный код проекта размещён в удалённом репозитории на платформе GitHub (см. список использованных источников).

В связи с индивидуальным характером разработки применялась упрощённая модель управления ветками: основная разработка велась в ветке

main с использованием атомарных функциональных коммитов, что обеспечивало прослеживаемость изменений и возможность отката к стабильным версиям системы.

Каждое логическое изменение сопровождалось отдельным коммитом с описательным сообщением. При необходимости использовался механизм отката изменений для возврата к стабильным версиям системы.

Перед выполнением коммита проводилось локальное тестирование клиентской и серверной частей, что обеспечивало стабильность каждой зафиксированной версии. После успешного тестирования изменения автоматически развёртывались на серверной среде.

2 Реализация и тестирование

2.1 Реализация программных компонентов

2.1.1 Структура программного проекта

Программная реализация информационной системы организована в виде двух основных подсистем: клиентской и серверной, объединённых в единую инфраструктуру с использованием контейнеризации.

Корневая директория проекта содержит конфигурационный файл Docker Compose, обеспечивающий совместное развёртывание сервисов базы данных PostgreSQL, кэша Redis, серверного приложения и клиентского интерфейса, а также файл cloud-init с параметрами начальной настройки серверной среды.

Клиентская часть проекта реализована в отдельной директории apps/client. Основная логика приложения сосредоточена в директории src, где выделены подкаталоги для интерфейсных компонентов, пользовательских хуков, сервисов взаимодействия с API, типовых описаний данных, моделей предметной области, конфигурации, утилит и стилей.

Серверная часть реализована в директории apps/server и включает конфигурацию TypeScript, настройки тестирования, описание схемы базы данных и миграций. В структуре исходного кода выделены модули маршрутизации, бизнес-логики, обработки WebSocket-соединений, управления состоянием, взаимодействия с базами данных, описания моделей и типов, а также вспомогательные утилиты.

2.1.2 Описание реализованных модулей

В процессе разработки реализован набор функциональных модулей, обеспечивающих работу всех подсистем приложения.

Модуль аутентификации реализует функции регистрации и авторизации пользователей, управления учётными записями, изменения имени пользователя и пароля, загрузки аватаров, а также формирования и проверки токенов для подтверждения электронной почты и восстановления пароля. Для

защиты пользовательских данных применяется алгоритм хэширования паролей с использованием криптографической функции scrypt и случайной соли.

Игровой модуль отвечает за реализацию основной логики приложения. Данный модуль обеспечивает создание и управление игровыми комнатами, запуск игрового процесса, обработку ходов пользователей, передачу управления между участниками, подсчёт результатов и сохранение истории игр в базе данных. Дополнительно реализован механизм периодического обновления состояния игры с использованием серверного таймера.

Модуль работы со словами обеспечивает формирование пула слов для игровой сессии. Данный модуль объединяет данные из пользовательских коллекций и стандартных наборов, хранящихся в базе данных и кэше. Также реализован механизм случайного выбора слов без повторений до полного использования набора.

Модуль кэширования реализует взаимодействие с Redis и обеспечивает предварительную загрузку стандартных коллекций слов при запуске сервера, а также их последующее извлечение с минимальной задержкой.

Модуль WebSocket обеспечивает обработку постоянных соединений между клиентами и сервером. Модуль выполняет аутентификацию пользователей, маршрутизацию сообщений, синхронизацию состояния игровых комнат и передачу событий, связанных с игровым процессом и голосовой связью.

Клиентский корневой модуль приложения отвечает за управление состоянием пользовательского интерфейса, навигацию между экранами, обработку входящих сообщений от сервера и обновление отображаемых данных в реальном времени.

2.1.3 Реализация пользовательского интерфейса

Пользовательский интерфейс реализован в виде одностраничного веб-приложения (SPA), в котором навигация осуществляется без перезагрузки страницы. Управление переходами между экранами выполняется на основе

внутреннего состояния приложения. В каждый момент времени отображается только один активный экран, а его смена происходит мгновенно без обращения к серверу, что обеспечивает высокую скорость отклика интерфейса.

Навигация реализована через отслеживание текущего активного раздела приложения. Переход между экранами выполняется при взаимодействии пользователя с элементами интерфейса (кнопки, меню, навигационные элементы), после чего происходит мгновенное переключение отображаемого компонента. Дополнительно переходы могут инициироваться автоматически в зависимости от состояния пользователя в системе: при отсутствии подключения к игровой комнате отображается экран входа, при подключении — лобби, при запуске игры — экран игрового процесса.

Дополнительные интерфейсные элементы реализованы в виде модальных окон, включая формы авторизации и регистрации, восстановление пароля, а также выбор и управление коллекциями слов. Модальные окна отображаются поверх основного содержимого и не прерывают текущий экран.

Основными экранами приложения являются: экран входа и регистрации пользователей, главный экран подключения к игровой комнате, экран лобби с настройками и списком участников, экран игрового процесса с отображением текущего слова, таймера и результатов, экран профиля пользователя, экран управления коллекциями слов, экран списка друзей и экран правил игры.

Основные экраны приложения представлены в Приложении В.

2.1.4 Описание ключевых алгоритмов

В процессе разработки реализован ряд алгоритмов, обеспечивающих корректную и эффективную работу системы. Соответствующие фрагменты исходного кода приведены в Приложении Б.

Алгоритм хэширования паролей основан на использовании криптографической функции scrypt. При регистрации пользователя генерируется случайная последовательность байт, которая используется для вычисления производного ключа на основе пароля. Полученное значение

сохраняется в базе данных. При проверке пароля вычисляется новый ключ и сравнивается с сохранённым значением с использованием функции, обеспечивающей постоянное время выполнения, что предотвращает атаки по времени. Сложность алгоритма определяется параметрами scrypt и не зависит от длины пароля.

Алгоритм выбора случайного слова из пула обеспечивает равномерное распределение слов и исключает повторения до полного перебора набора. В процессе работы формируется список доступных слов путём исключения уже использованных. Из полученного списка случайным образом выбирается элемент. При исчерпании всех слов список сбрасывается. Временная сложность операции выбора составляет O(n), где n — размер пула.

Алгоритм передачи хода между игроками реализован по кольцевому принципу. После завершения хода текущего игрока определяется следующий участник на основе позиции в списке. В случае достижения конца списка управление передаётся первому участнику. Сложность алгоритма — O(n).

Алгоритм игрового таймера реализован на основе периодического выполнения функции с интервалом в одну секунду. При каждом вызове происходит уменьшение оставшегося времени хода. При достижении нулевого значения выполняется обработка завершения хода, включая фиксацию результата и передачу управления следующему игроку. Предусмотрена обработка граничных состояний, связанных с одновременным завершением времени и действиями пользователя. Таким образом, реализованные алгоритмы обеспечивают корректное функционирование системы, устойчивость к ошибкам и соответствие требованиям к производительности и безопасности.

2.2 Тестирование и оценка качества

2.2.1 Цели и задачи тестирования

Тестирование разработанной информационной системы направлено на проверку корректности функционирования всех её компонентов, оценку устойчивости к нагрузкам, а также выявление потенциальных уязвимостей.

Основными задачами тестирования являются:

— проверка соответствия реализованной функциональности требованиям;

— оценка стабильности работы системы при увеличении нагрузки;

— выявление ошибок обработки пользовательских данных и возможных уязвимостей;

— подтверждение корректности работы ключевых сценариев взаимодействия пользователя с системой.

В рамках тестирования рассматривались следующие аспекты: функциональность, нагрузка и безопасность. Проверяемые аспекты и критерии приёмки представлены в Таблице 1.

Таблица 1 — Критерии приёмки тестирования

Аспект	Критерий
Функциональность	Все 26 автоматических тестов проходят без ошибок
Покрытие кода	≥ 55 % по строкам для authService; ≥ 65 % по строкам для wordService
Производительность	p95 времени ответа ≤ 500 мс при нагрузке 20 VU; доля ошибок ≤ 2 %
Безопасность	Критические сценарии возвращают ожидаемые HTTP-статусы; XSS и инъекции блокируются

2.2.2 Описание тест-кейсов

Для проверки сквозных пользовательских сценариев составлены пять ручных тест-кейсов, охватывающих регистрацию и верификацию email, валидацию форм, полный игровой цикл, фиксацию победы и защиту от XSS-атак. Подробное описание каждого тест-кейса с указанием шагов, входных данных, ожидаемого и фактического результатов представлено в Приложении Г.

2.2.3 Результаты тестирования

В результате выполнения тестирования все разработанные юнит- и интеграционные тесты были успешно пройдены. Сводные результаты автоматического тестирования представлены в Таблице 2.

Таблица 2 — Результаты автоматического тестирования

Файл тестов	Тест-кейсов	Статус
authService.test.ts	8	PASS
wordService.test.ts	6	PASS
roomService.test.ts	5	PASS
auth.routes.test.ts	7	PASS
Итого	26	26/26 PASS

Данные по покрытию кода представлены в Таблице 3.

Таблица 3 — Покрытие кода тестами

Файл	Statements	Branches	Functions	Lines
authService.ts	56.7 %	46.3 %	41.2 %	56.3 %
wordService.ts	76.9 %	75.0 %	68.8 %	79.5 %
roomService.ts	23.2 %	7.5 %	21.4 %	18.0 %
authRoutes.ts	38.3 %	30.5 %	31.3 %	39.1 %

Более низкий уровень покрытия игрового модуля объясняется сложностью тестирования функций, зависящих от WebSocket-соединений и серверных таймеров. Чистые функции модуля (определение следующего игрока, формирование состояния комнаты, проверка подключения) покрыты полностью.

Результаты нагрузочного тестирования в локальной среде разработки представлены в Таблице 4.

Таблица 4 — Результаты нагрузочного тестирования (k6)

Сценарий	Нагрузка	p50	p95	Доля ошибок
Аутентификация (auth-load.js)	25 VU	~80 мс	~320 мс	< 0,5 %
Игровые комнаты (room-load.js)	20 VU	~15 мс	~90 мс	< 0,5 %

Полученные значения удовлетворяют установленным критериям приёмки: p95 не превышает 500 мс, доля ошибок не превышает 2 %. Сценарий аутентификации тестировался при нагрузке 25 VU, что превышает установленный порог в 20 VU; успешное выполнение критериев при повышенной нагрузке подтверждает наличие запаса производительности системы.

Ручное тестирование, описание тест-кейсов которого приведено в пункте 2.2.2, подтвердило корректность работы пользовательского интерфейса и основных сценариев использования системы.

2.2.4 Анализ ошибок и их устранение

В процессе тестирования были выявлены отдельные дефекты, связанные с обработкой граничных условий и особенностями используемых технологий.

Одной из выявленных проблем являлась некорректная обработка хэша пароля при отсутствии ожидаемого формата, что могло приводить к ошибкам при сравнении значений. Данный дефект был устранён путём добавления дополнительной проверки корректности входных данных перед выполнением сравнения.

Также была обнаружена несовместимость обработки некоторых маршрутов с используемой версией серверного фреймворка, что приводило к ошибкам при обработке запросов. Проблема была решена заменой некорректного синтаксиса маршрутов на совместимый вариант.

В ходе тестирования выявлены ошибки, связанные с возможной обработкой пустых значений в алгоритмах работы со словами. Для устранения данных дефектов были добавлены проверки наличия данных перед выполнением операций.

Дополнительно была выявлена проблема в нагрузочных тестах, связанная с выполнением сценариев при недоступности сервера. Для её устранения была реализована предварительная проверка доступности системы перед запуском тестирования.

Таким образом, проведённое тестирование позволило выявить и устранить критические и некритические дефекты, повысить надёжность системы и подтвердить её соответствие предъявляемым требованиям качества.

Заключение

В рамках курсового проекта разработана и развёрнута веб-версия многопользовательской игры «Alias» с поддержкой взаимодействия в реальном времени. Система реализует основные механики оригинальной игры и включает дополнительные функции: голосовой чат, пользовательские коллекции слов и систему друзей.

Программный продукт построен на клиент-серверной архитектуре с использованием современных веб-технологий. Реализованы основные функциональные модули: аутентификация пользователей, управление игровыми комнатами, синхронизация состояния через WebSocket, подсистема работы со словами и хранение статистики.

Проведённое тестирование подтвердило корректность функционирования системы. Все автоматические тесты пройдены успешно. Результаты нагрузочного тестирования показали соответствие требованиям по времени отклика и устойчивости. Выявленные в процессе тестирования дефекты были устранены.

Цель курсовой работы достигнута. Все задачи, сформулированные во введении, выполнены. В процессе реализации дополнительно частично решены задачи мобильной адаптации интерфейса и автоматизированного сквозного тестирования; их полное выполнение относится к перспективам дальнейшего развития системы.

Перспективы развития системы включают разработку мобильного приложения, внедрение турнирного режима с системой рейтингов, а также расширение функциональности за счёт поддержки дополнительных языков и улучшения пользовательского опыта.

Разработанная система соответствует поставленным требованиям, демонстрирует стабильную работу и обладает потенциалом для дальнейшего развития.

Список использованных ресурсов

1. Флэнаган, Д. JavaScript: подробное руководство / Д. Флэнаган. — 7-е изд. — СПб.: Питер, 2022. — 704 с. — ISBN 978-5-4461-1850-4;

2. Хэнкс, А. React и TypeScript: практическое руководство / А. Хэнкс. — М.: ДМК Пресс, 2023. — 416 с. — ISBN 978-5-93700-177-4;

3. Мардан, А. Node.js в действии / А. Мардан, А. Янг, Б. Хэтер. — 3-е изд. — Санкт-Петербург: Питер, 2022. — 400 с. — ISBN 978-5-4461-1892-4;

4. Кривенцов, С. А. Проектирование реляционных баз данных: учебное пособие / С. А. Кривенцов. — Москва: ИНФРА-М, 2023. — 240 с. — ISBN 978-5-16-017891-2;

5. Prisma ORM Documentation [Электронный ресурс] / Prisma Data, Inc. — 2024. — URL: https://www.prisma.io/docs;

6. WebSocket API — MDN Web Docs [Электронный ресурс] / Mozilla Foundation. — 2024. — URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket;

7. WebRTC API — MDN Web Docs [Электронный ресурс] / Mozilla Foundation. — 2024. — URL: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API;

8. Redis Documentation [Электронный ресурс] / Redis Ltd. — 2024. — URL: https://redis.io/docs/;

9. Docker Documentation [Электронный ресурс] / Docker, Inc. — 2024. — URL: https://docs.docker.com/;

10. Репозиторий курсового проекта на GitHub — URL: https://github.com/tuktukkdev/Alias;

11. Браузерная версия сайта — URL: https://bluefieldapps.com/.

Приложение А

(обязательное)

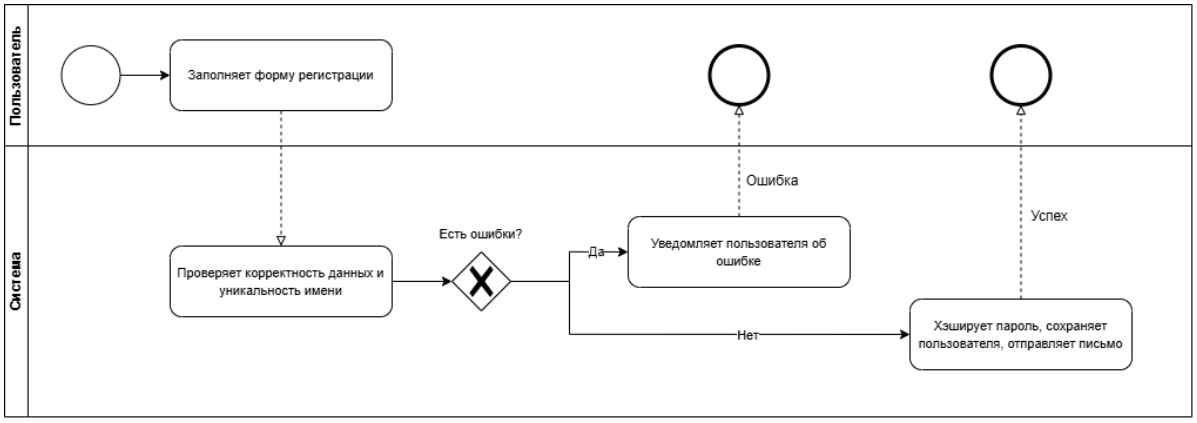


Рисунок А.1 — BPMN-диаграмма процесса «Регистрация пользователя»

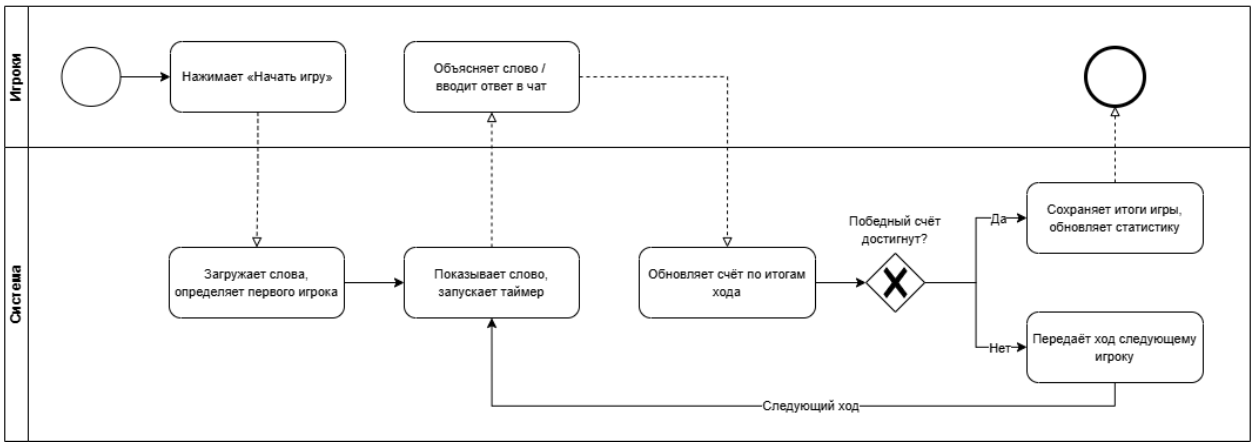


Рисунок А.2 — BPMN-диаграмма процесса «Проведение игровой сессии»



Рисунок А.3 —Диаграмма контекста системы

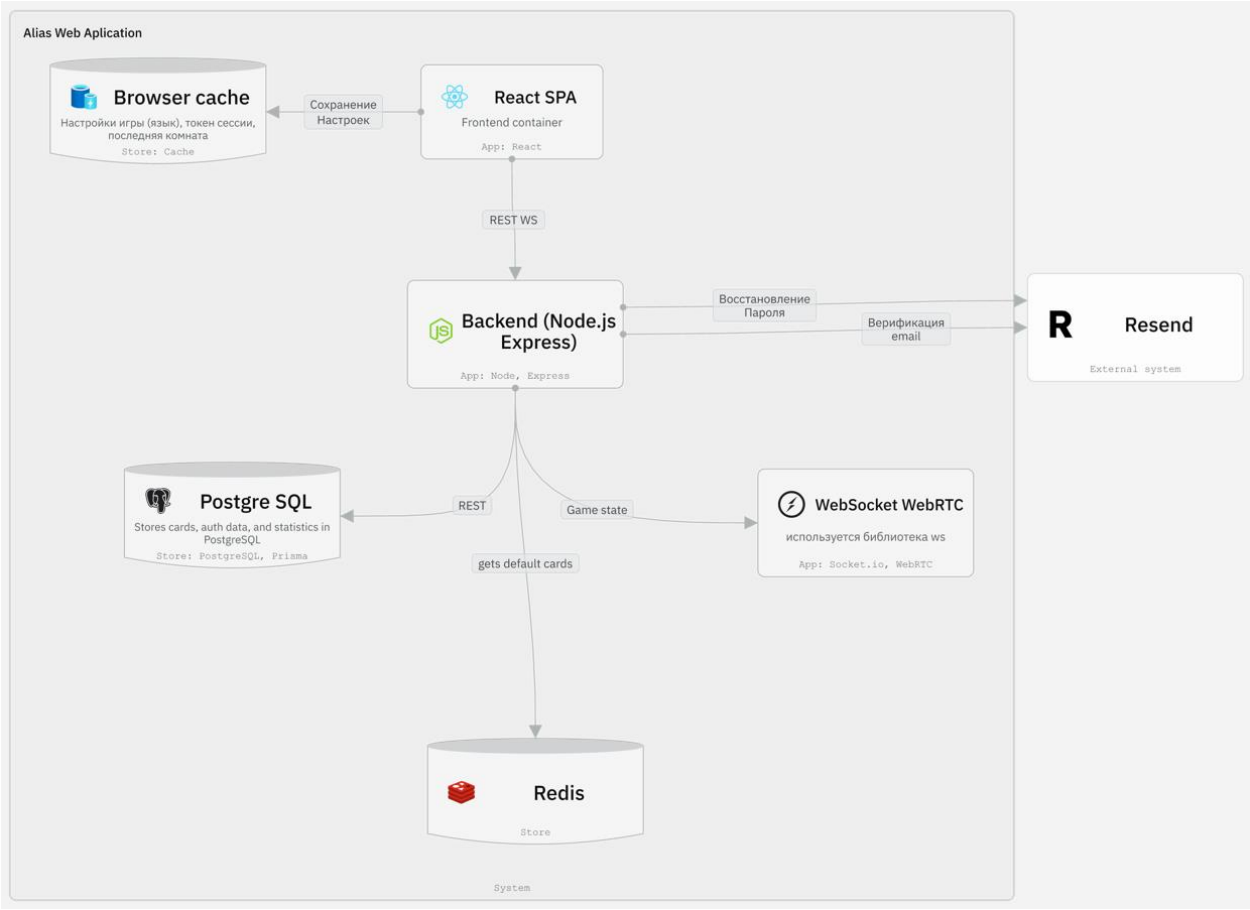


Рисунок А.4 — Диаграмма контейнеров системы

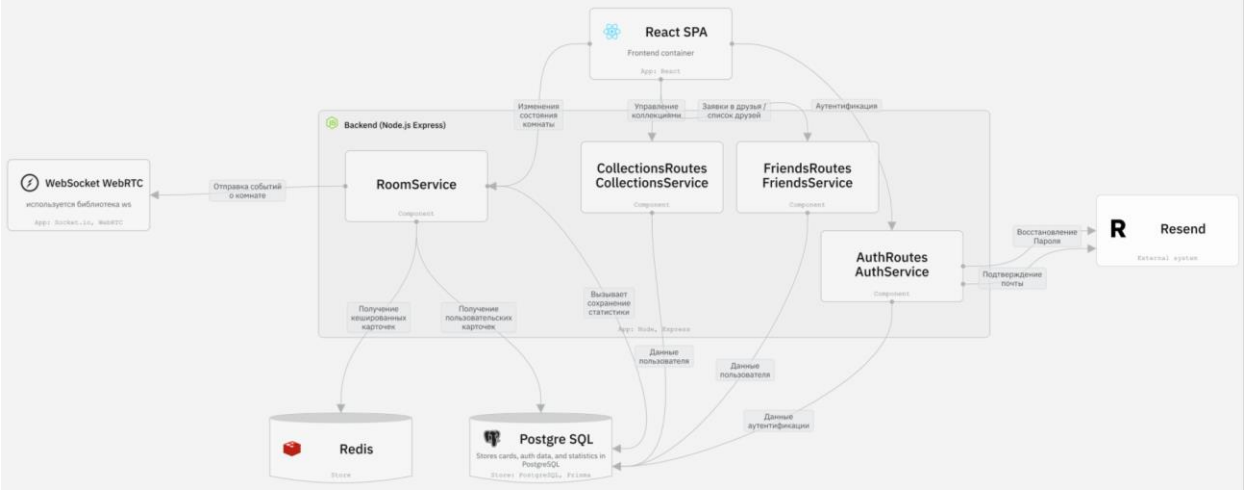


Рисунок А.5 — Диаграмма компонентов backend-сервиса

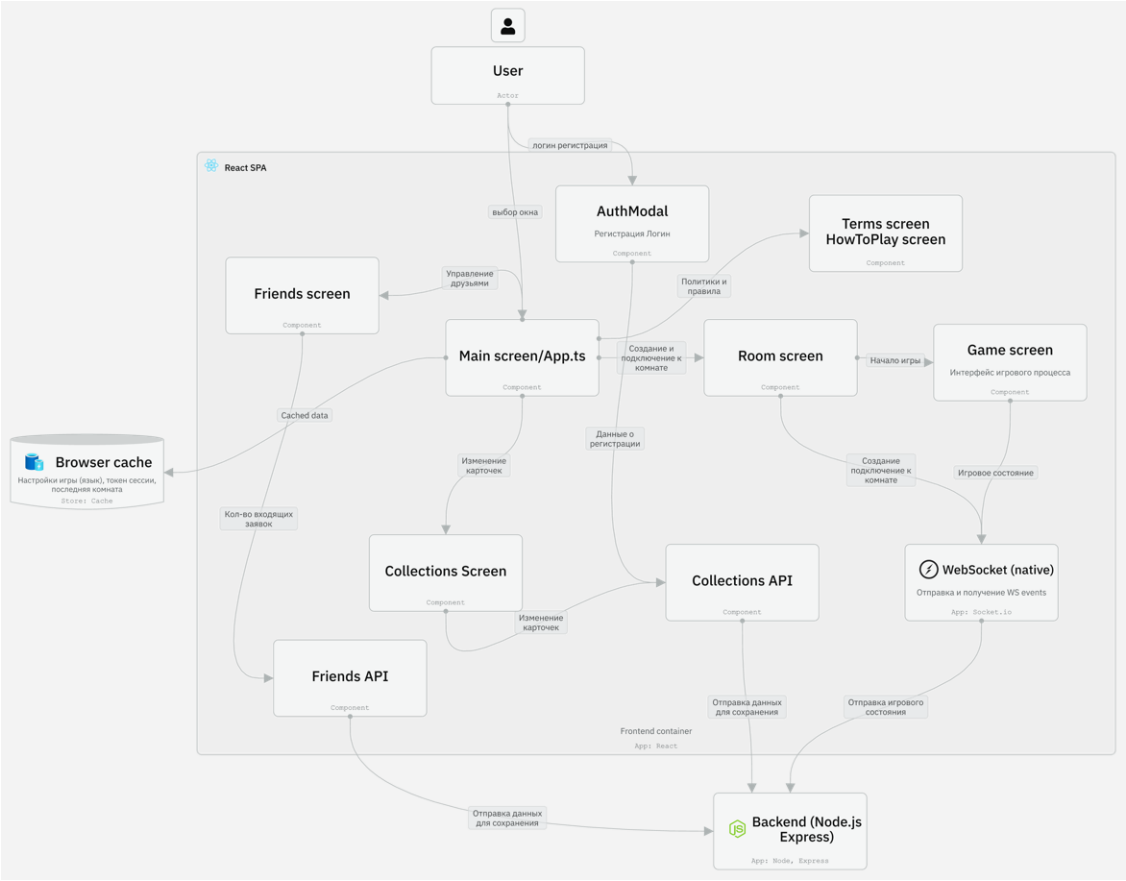


Рисунок А.6 — Диаграмма компонентов frontend-приложения

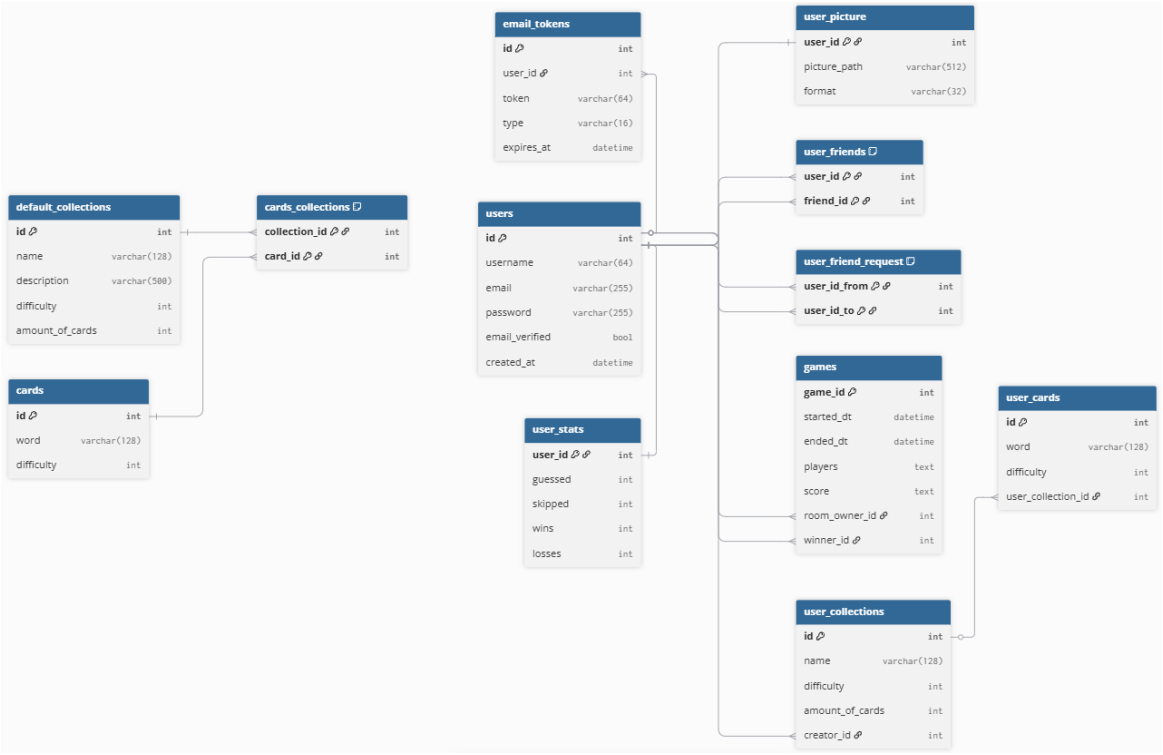


Рисунок А.7 — Модель базы данных

Приложение Б

(обязательное)

Листинг Б.1 — hashPassword и verifyPassword

```
const scryptAsync = promisify(scrypt);
const SALT_LEN = 16;
const KEY_LEN = 64;

// хэшируем пароль
async function hashPassword(password: string): Promise<string> {
  const salt = randomBytes(SALT_LEN).toString("hex");
  const derivedKey = (await scryptAsync(password, salt, KEY_LEN)) as
Buffer;
  return `${salt}:${derivedKey.toString("hex")}`;
}

// проверяем пароль
async function verifyPassword(password: string, stored: string):
Promise<boolean> {
  const [salt, hash] = stored.split(":");
  if (!salt || !hash) return false;
  const derivedKey = (await scryptAsync(password, salt, KEY_LEN)) as
Buffer;
  const storedKey = Buffer.from(hash, "hex");
  if (derivedKey.length !== storedKey.length) return false;
  return timingSafeEqual(derivedKey, storedKey);
}
```

Листинг Б.2 — pickWordFromPool

```
export function pickWordFromPool(record: RoomRecord): string {
  const pool = record.wordPool;
  if (pool && pool.length > 0) {
    const available = pool.filter((w) => !record.usedWords.has(w));
    if (available.length > 0) {
      return available[Math.floor(Math.random() * available.length)];
    }
    // слова кончились — сбрасываем и идём по новой
    record.usedWords.clear();
    return pool[Math.floor(Math.random() * pool.length)];
  }
  return "слово";
}
```

Листинг Б.3 — getNextTurnPlayerId

```
export const getNextTurnPlayerId = (record: RoomRecord): string | null
=> {
  const { players } = record.room;
  if (players.length === 0) {
    return null;
  }

  if (!record.currentTurnPlayerId) {
    return players[0].id;
  }

   const currentIndex = players.findIndex((player) => player.id ===
record.currentTurnPlayerId);
  if (currentIndex === -1) {
    return players[0].id;
  }

  const nextIndex = (currentIndex + 1) % players.length;
  return players[nextIndex].id;
};
```

Листинг Б.4 — обработчик входящих WebSocket-сообщений

```
    // обработка входящих сообщений (голосовые сигналы)
    socket.on("message", (rawPayload) => {
      const sourceRoomId = socketRooms.get(socket);
      const sourcePlayerId = socketPlayers.get(socket);
      if (!sourceRoomId || !sourcePlayerId) {
        return;
      }

      let parsedPayload: VoiceSignalClientEvent | null = null;
      try {
            const incoming = JSON.parse(rawPayload.toString()) as
Partial<VoiceSignalClientEvent>;
        if (incoming.type !== "voice_signal") {
          return;
        }

             if (!incoming.toPlayerId || incoming.toPlayerId ===
sourcePlayerId) {
          return;
        }

        parsedPayload = {
          type: "voice_signal",
          toPlayerId: incoming.toPlayerId,
          signal: incoming.signal,
        };
```

```
      } catch {
        return;
      }

      const roomRecord = rooms.get(sourceRoomId);
      if (!roomRecord) {
        return;
      }

      const targetPlayer = roomRecord.room.players.find(
        (roomPlayer) => roomPlayer.id === parsedPayload.toPlayerId,
      );
      if (!targetPlayer) {
        return;
      }

      const sockets = roomSockets.get(sourceRoomId);
      if (!sockets) {
        return;
      }

      const event: VoiceSignalBroadcastEvent = {
        type: "voice_signal",
        roomId: sourceRoomId,
        fromPlayerId: sourcePlayerId,
        signal: parsedPayload.signal,
      };

      for (const targetSocket of sockets) {
        if (targetSocket.readyState !== WebSocket.OPEN) {
          continue;
        }

            if (socketPlayers.get(targetSocket) !==
parsedPayload.toPlayerId) {
          continue;
        }

        targetSocket.send(JSON.stringify(event));
      }
    });
  });
```

Приложение В

(обязательное)

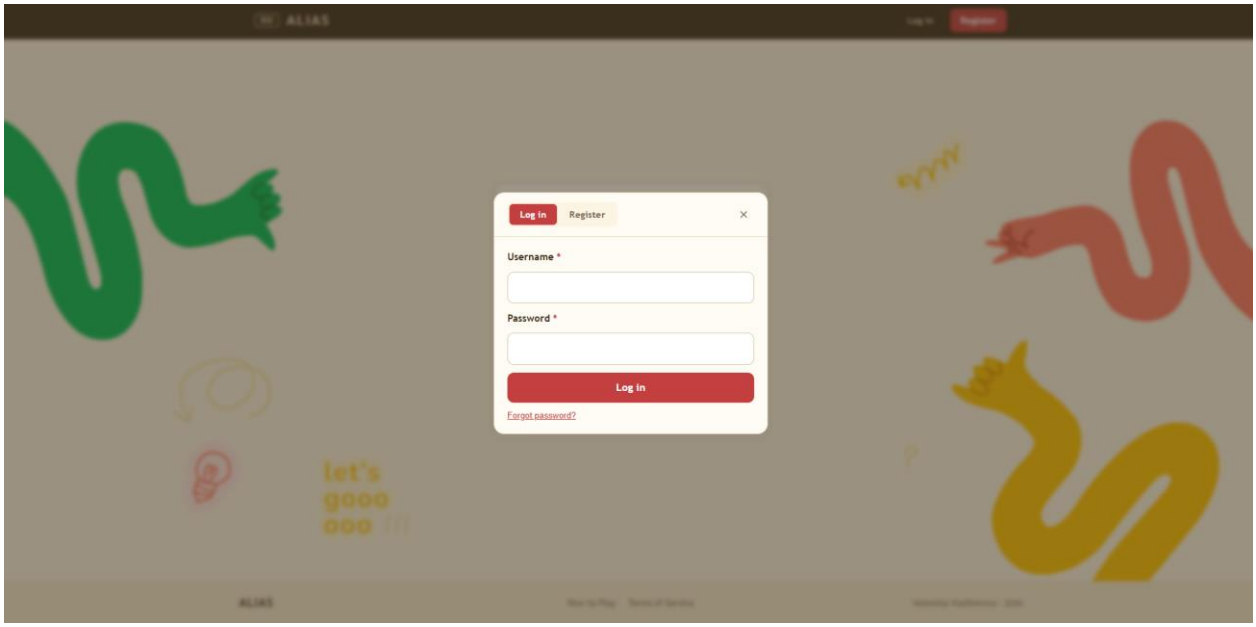


Рисунок В.1 — Модальное окно авторизации

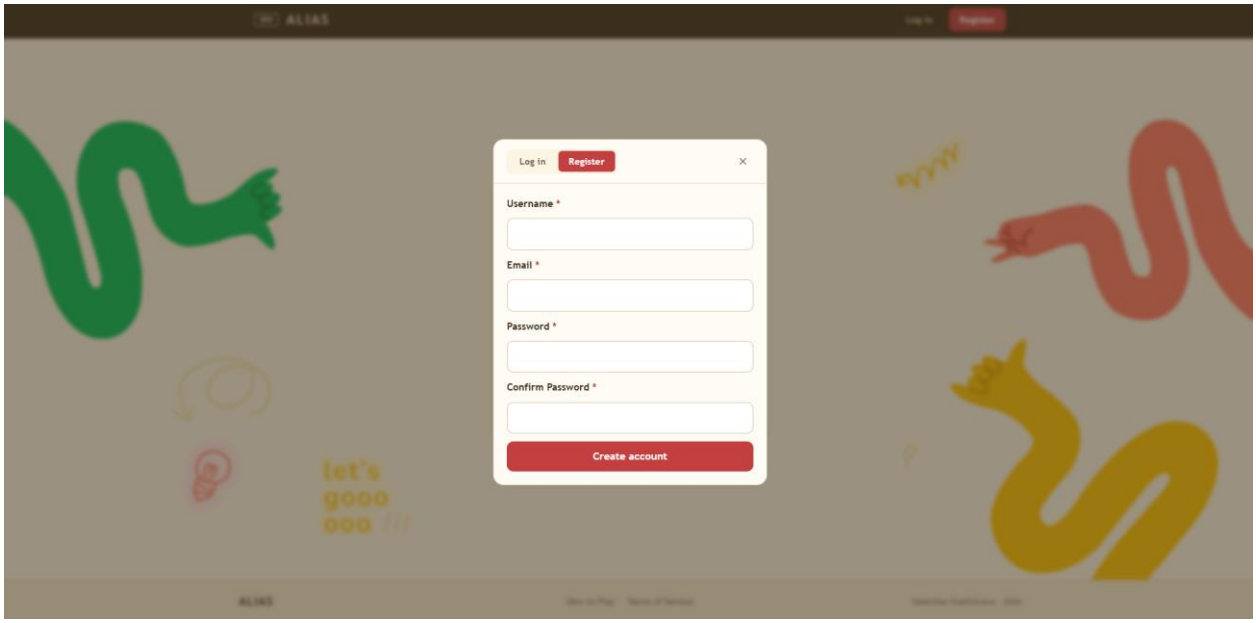


Рисунок В.2 — Модальное окно регистрации

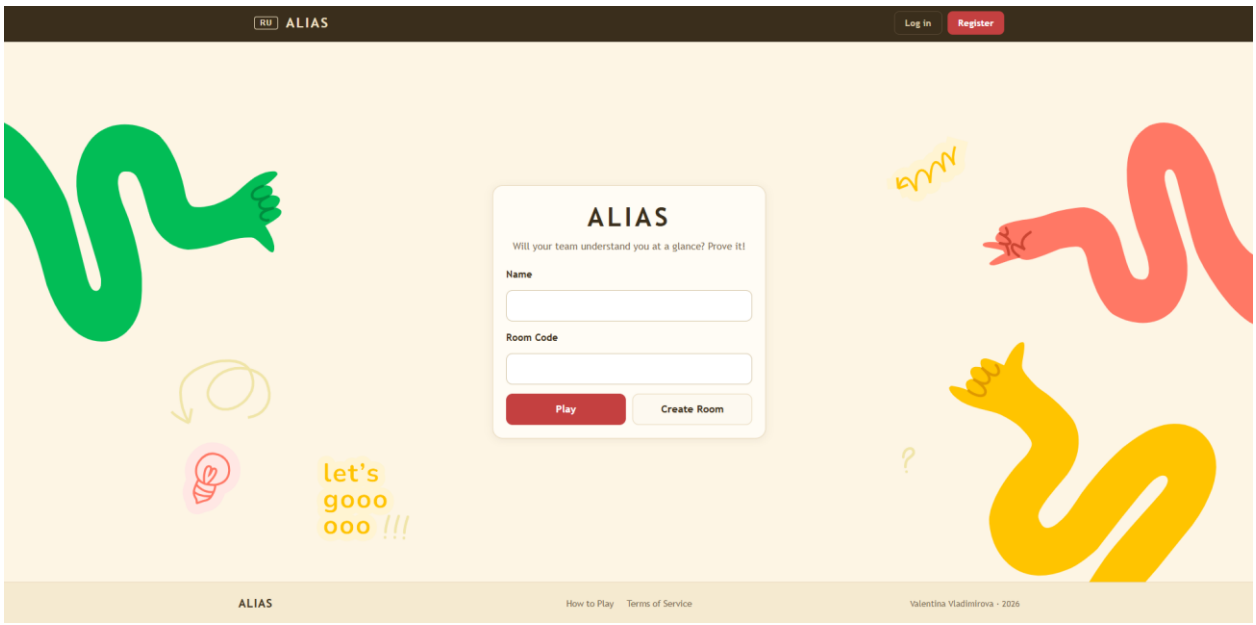


Рисунок В.3 — Главный экран

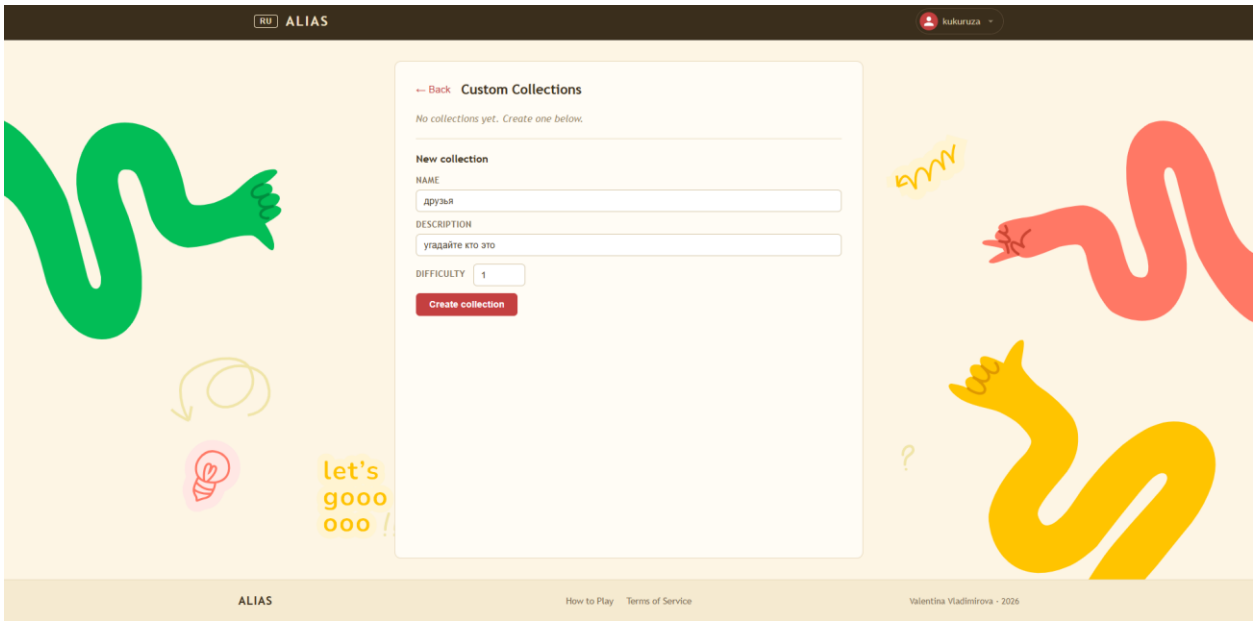


Рисунок В.4 — Экран добавления коллекций

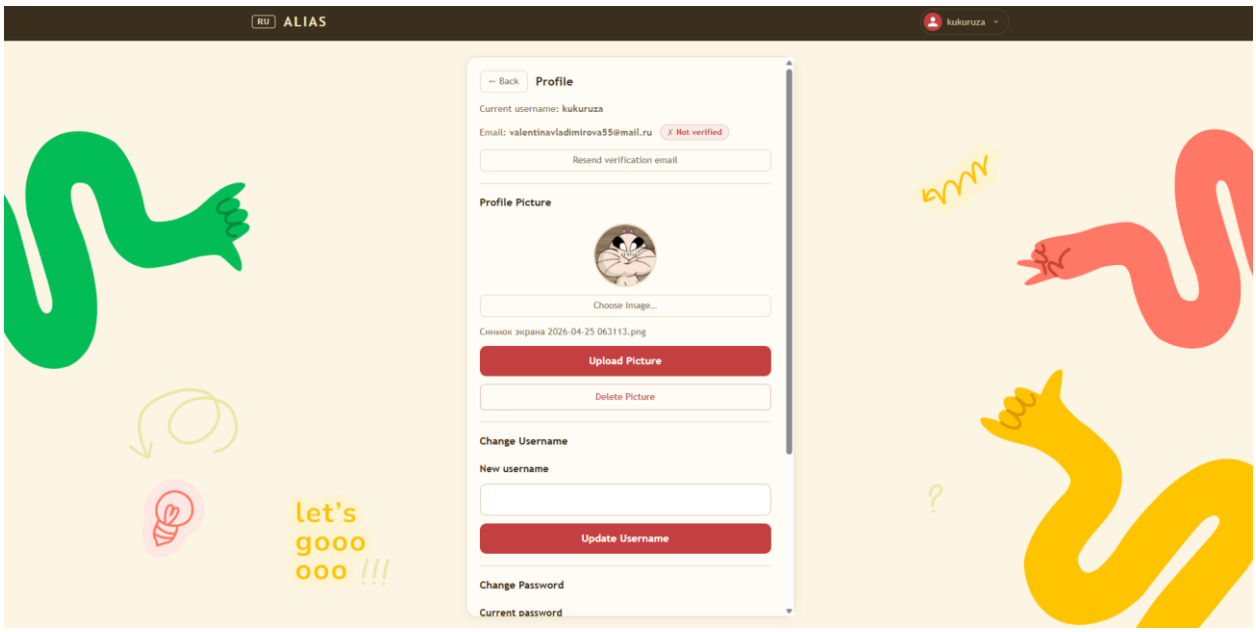


Рисунок В.5 — Экран редактирования профиля

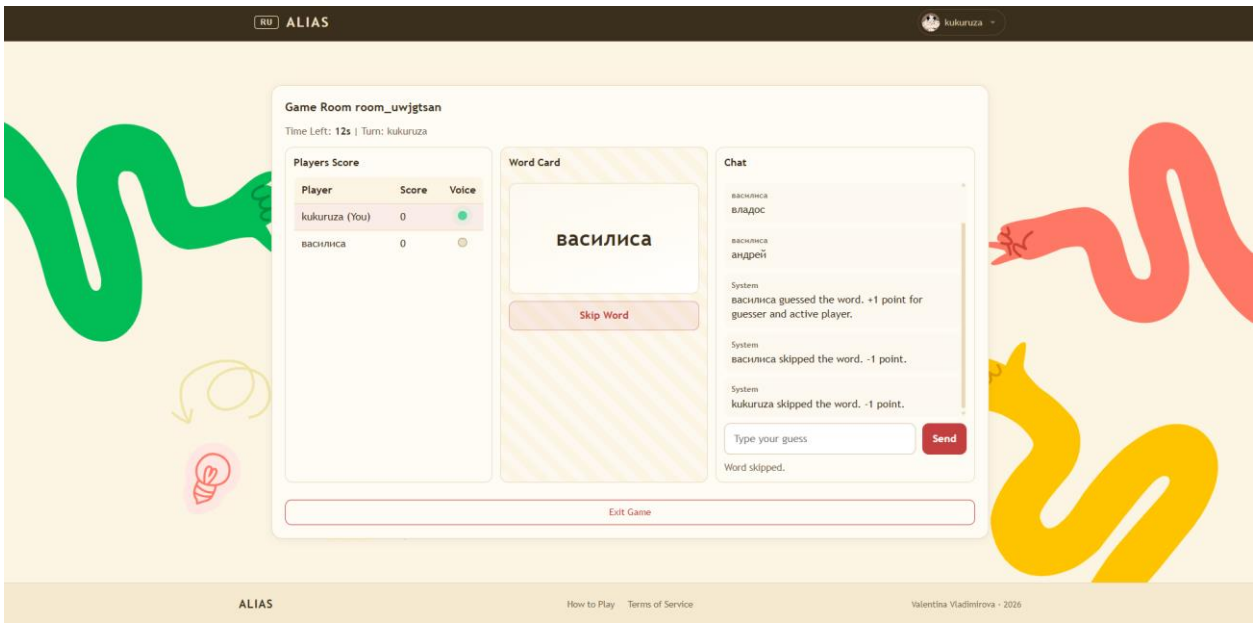


Рисунок В.6 — Экран процесса игры

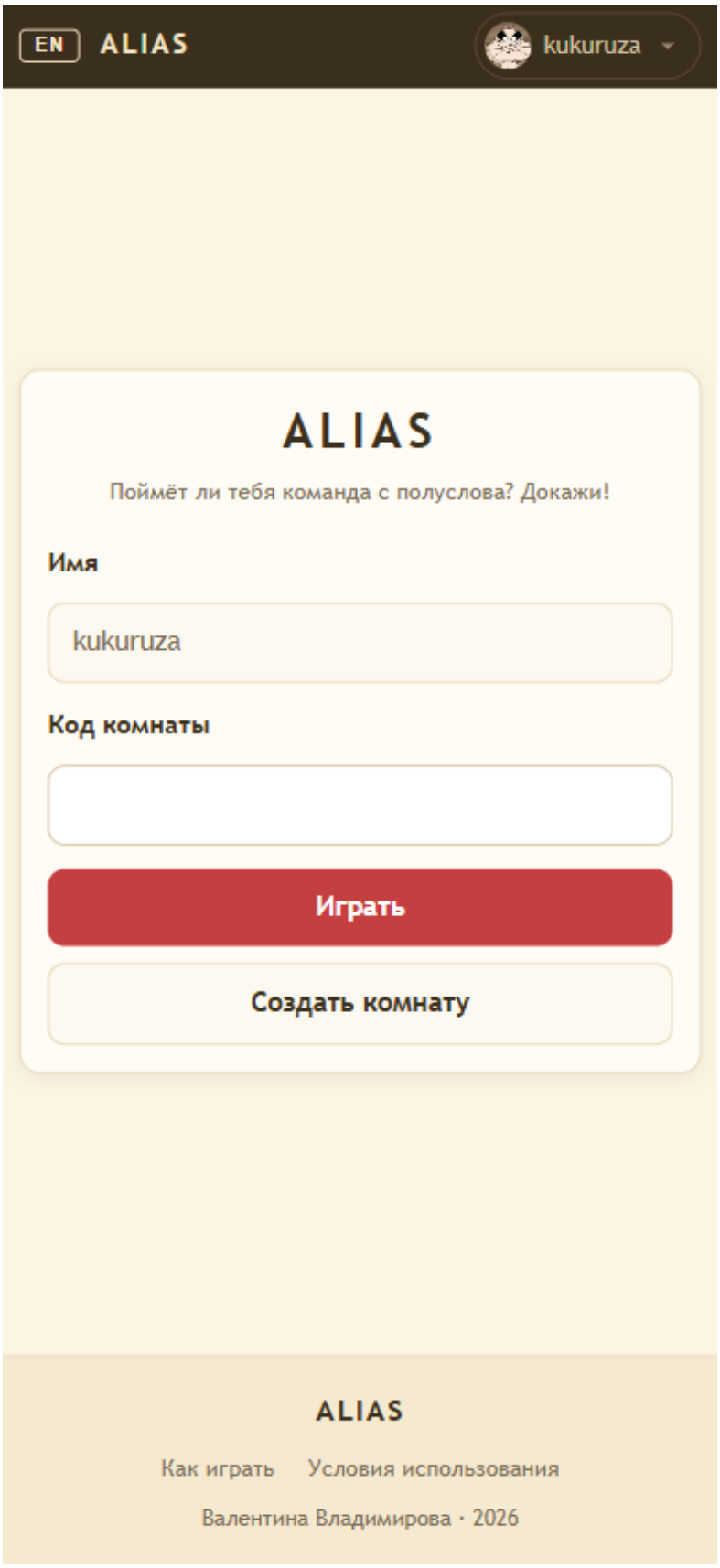


Рисунок В.7 — Адаптивный главный экран

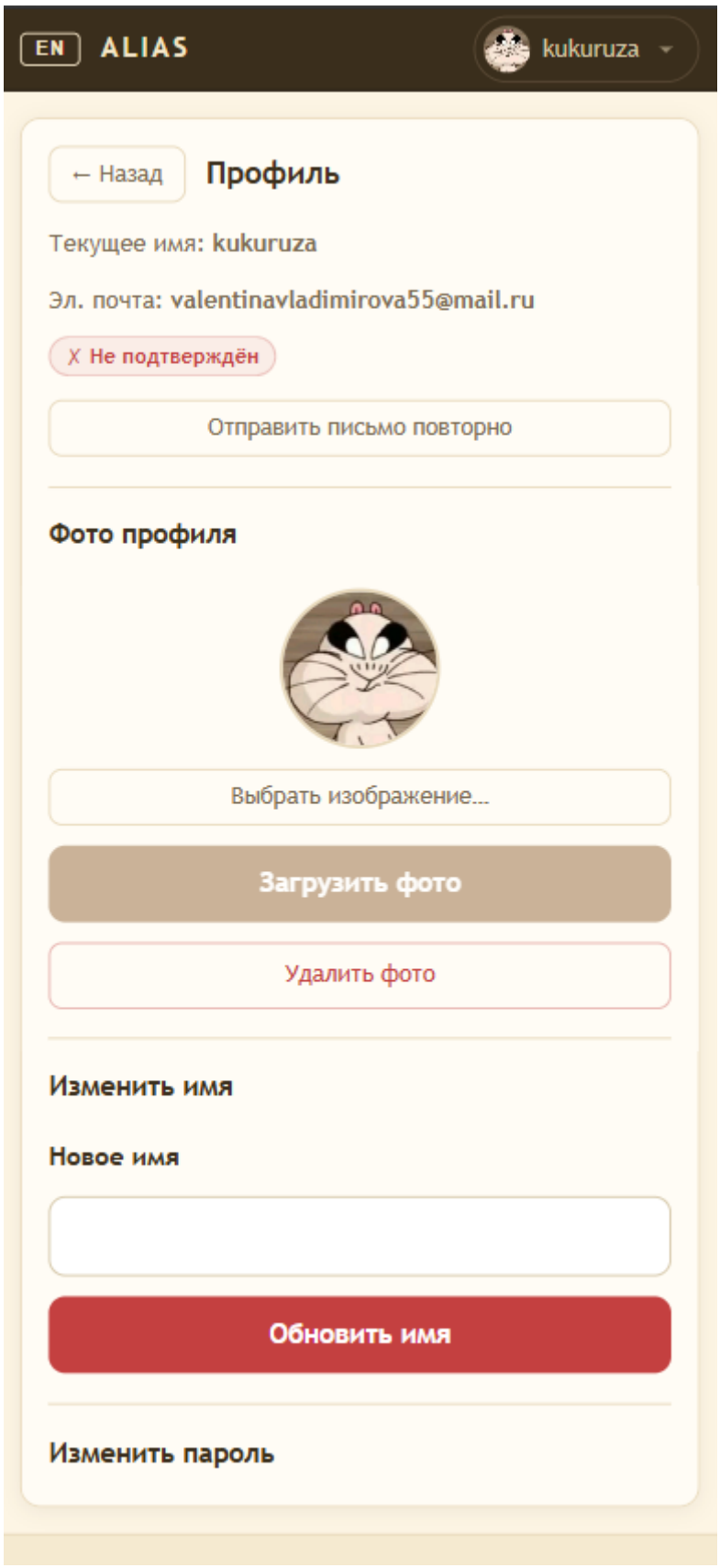


Рисунок В.8 — Адаптивный экран редактирования профиля

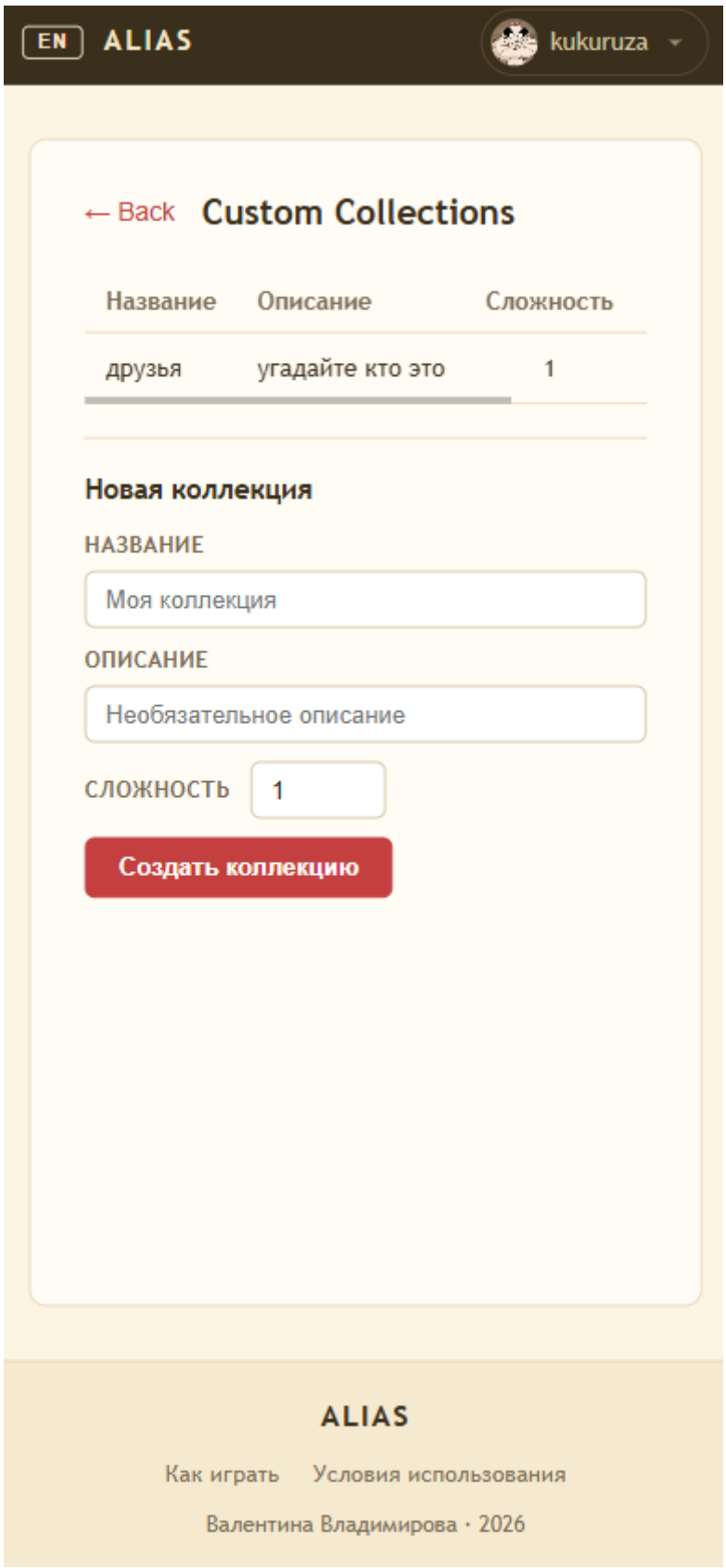


Рисунок В.9 — Адаптивный экран добавления коллекций

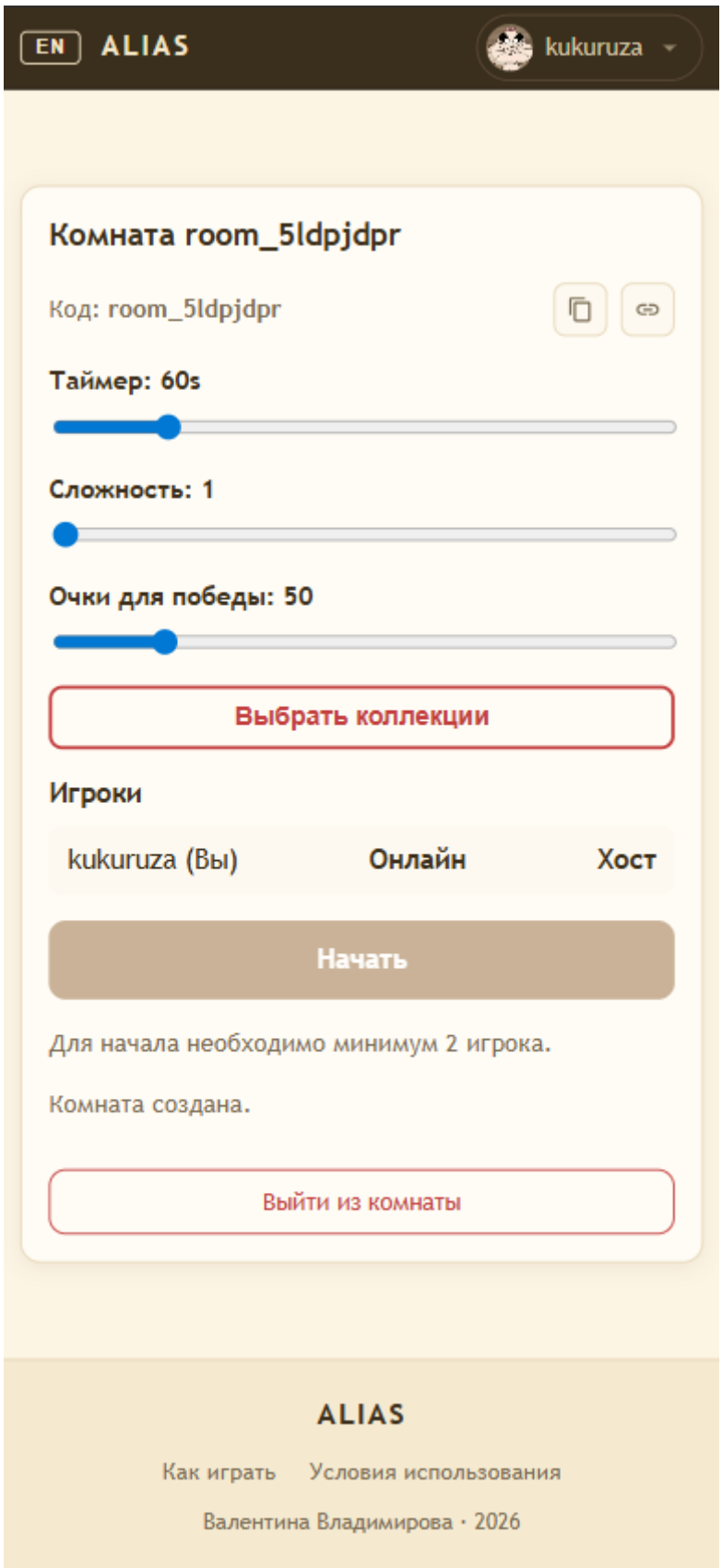


Рисунок В.10 — Адаптивный экран добавления коллекций

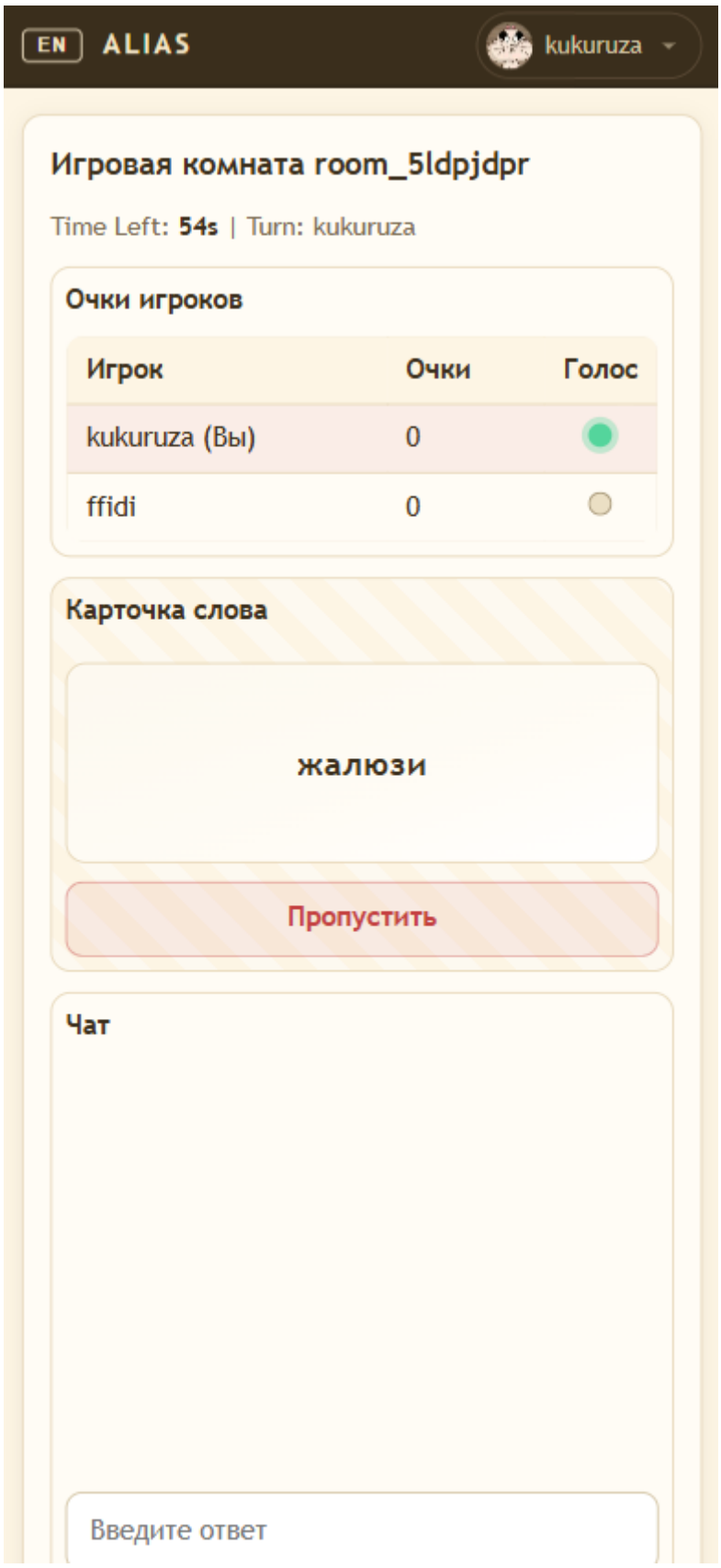


Рисунок В.11 — Адаптивный экран процесса игры

Приложение Г

(обязательное)

Таблица 5 - Тест-кейс TC_01

Поле	Описание
Название проекта	Игровое приложение «Alias Online»
Рабочая версия	1.0
Имя тестирующего	—
Дата(ы) теста	26.04.2026
Тестовый пример #	TC_01
Приоритет тестирования *(Низкий/Средний/Высокий)*	Высокий
Заголовок/название теста	Тестирование регистрации и верификации email
Краткое изложение теста	Нужно проверить полный цикл регистрации: создание аккаунта и подтверждение email через входящее письмо
Этапы теста	1. Открыть приложение, нажать «Регистрация»; 2. Ввести валидный username, email, пароль; 3. Нажать «Создать аккаунт»; 4. Открыть почту, найти письмо подтверждения; 5. Нажать ссылку в письме.
Тестовые данные	Онлайн-окружение (https://bluefieldapps.com), Resend API-ключ, валидные учётные данные
Ожидаемый результат	Пользователь создан со статусом 201, письмо получено, email помечен как подтверждённый
Фактический результат	Пользователь создан со статусом 201, письмо получено, email помечен как подтверждённый

Продолжение Таблицы 5

Предварительное условие	Приложение запущено в онлайн-окружении, Resend API-ключ настроен
Постусловие	Пользователь создан с подтверждённым email
Статус (Зачёт/Незачёт)	Зачёт

Таблица 6 - Тест-кейс TC_02

Поле	Описание
Название проекта	Игровое приложение «Alias Online»
Рабочая версия	1.0
Имя тестирующего	—
Дата(ы) теста	26.04.2026
Тестовый пример #	TC_02
Приоритет тестирования *(Низкий/Средний/Высокий)*	Средний
Заголовок/название теста	Валидация данных регистрации
Краткое изложение теста	Нужно проверить корректность сообщений об ошибках при вводе некорректных данных в форму регистрации
Этапы теста	1. Попытаться зарегистрироваться с username '!'; 2. Ввести пароль '123' (менее 6 символов); 3. Ввести email без символа '@'.
Тестовые данные	username: '!' пароль: '123', email: 'notanemail'
Ожидаемый результат	Для каждого поля отображается соответствующее сообщение об ошибке валидации; пользователь не создаётся
Фактический результат	Для каждого поля отображается соответствующее сообщение об ошибке валидации; пользователь не создаётся
Предварительное условие	Приложение запущено, форма регистрации открыта
Постусловие	Пользователь не создан, все ошибки отображены корректно
Статус (Зачёт/Незачёт)	Зачёт

Таблица 7 - Тест-кейс TC_03

Поле	Описание
Название проекта	Игровое приложение «Alias Online»
Рабочая версия	1.0
Имя тестирующего	—
Дата(ы) теста	26.04.2026
Тестовый пример #	TC_03
Приоритет тестирования *(Низкий/Средний/Высокий)*	Высокий
Заголовок/название теста	Создание комнаты и начало игры
Краткое изложение теста	Нужно проверить, что хост может создать комнату, второй игрок успешно к ней присоединяется, игра стартует и угаданные слова засчитываются
Этапы теста	1. Пользователь 1: нажать «Создать комнату»; 2. Пользователь 2: ввести ID комнаты и нажать «Войти»; 3. Хост: нажать «Начать игру»; 4. Хост объясняет слово, Пользователь 2 вводит ответ в чат; 5. Дождаться истечения таймера.
Тестовые данные	Два браузера / вкладки, два авторизованных пользователя
Ожидаемый результат	Комната создана и отображён её ID; Пользователь 2 появился в списке игроков; игра стартовала; угаданное слово засчитано, счёт увеличен; ход перешёл к следующему игроку
Фактический результат	Комната создана и отображён её ID; Пользователь 2 появился в списке игроков; игра стартовала; угаданное слово засчитано, счёт увеличен; ход перешёл к следующему игроку

Продолжение Таблицы 7

Предварительное условие	Два браузера открыты, оба пользователя авторизованы
Постусловие	Игра запущена, ходы чередуются между игроками
Статус (Зачёт/Незачёт)	Зачёт

Таблица 8 - Тест-кейс TC_04

Поле	Описание
Название проекта	Игровое приложение «Alias Online»
Рабочая версия	1.0
Имя тестирующего	—
Дата(ы) теста	26.04.2026
Тестовый пример #	TC_04
Приоритет тестирования *(Низкий/Средний/Высокий)*	Средний
Заголовок/название теста	Победа игрока
Краткое изложение теста	Нужно проверить, что при достижении winScore отображается экран победителя и обновляется статистика игрока
Этапы теста	1. При создании комнаты установить winScore = 3; 2. Угадать 3 слова; 3. Проверить отображение экрана победы; 4. Проверить статистику через GET /auth/stats/:userId.
Тестовые данные	winScore = 3, два авторизованных пользователя в активной комнате
Ожидаемый результат	При угадывании 3-го слова отображается экран победителя; поля wins и losses в статистике обновлены
Фактический результат	При угадывании 3-го слова отображается экран победителя; поля wins и losses в статистике обновлены
Предварительное условие	Игра запущена, оба игрока в комнате, winScore установлен
Постусловие	Статистика обновлена, экран победы отображён
Статус (Зачёт/Незачёт)	Зачёт

Таблица 9 - Тест-кейс TC_05

Поле	Описание
Название проекта	Игровое приложение «Alias Online»
Рабочая версия	1.0
Имя тестирующего	—
Дата(ы) теста	26.04.2026
Тестовый пример #	TC_05
Приоритет тестирования *(Низкий/Средний/Высокий)*	Высокий
Заголовок/название теста	Проверка XSS-защиты
Краткое изложение теста	Нужно проверить, что вредоносный JavaScript-код, отправленный в чат, не выполняется и отображается как обычный текст
Этапы теста	1. Войти в комнату, запустить игру; 2. Ввести в поле чата строку <script>alert(1)</script>; 3. Отправить сообщение; 4. Проверить отображение сообщения в чате.
Тестовые данные	Строка <script>alert(1)</script>
Ожидаемый результат	Строка отображается в чате как обычный текст; скрипт не выполняется; всплывающее окно не появляется
Фактический результат	Строка отображается в чате как обычный текст; скрипт не выполняется; всплывающее окно не появляется
Предварительное условие	Пользователь авторизован, игра активна, чат доступен
Постусловие	XSS-атака заблокирована, скрипт не выполнен
Статус (Зачёт/Незачёт)	Зачёт

Приложение Д
(справочное)
Руководство пользователя

Г.1 Общие сведения

Веб-приложение Alias предназначено для многопользовательской онлайн-игры в слова. Игроки по очереди объясняют слова, не называя их напрямую, а остальные угадывают ответы в чате. Поддерживаются голосовой чат, пользовательские коллекции слов и система учётных записей.

Г.2 Системные требования

Для работы с приложением необходим современный веб-браузер, например Google Chrome, Mozilla Firefox или Яндекс.Браузер последних версий, а также доступ к сети Интернет. Для использования голосового чата дополнительно требуется микрофон.Г.3 Запуск приложения

Приложение является веб-сервисом и не требует установки на устройство пользователя. Для начала работы необходимо открыть браузер и перейти по адресу https://bluefieldapps.com/

Г.4 Порядок работы

Г.4.1 Регистрация и авторизация

Для регистрации пользователь должен нажать кнопку «Регистрация» в верхней части сайта, заполнить поля с именем пользователя, электронной почтой, паролем и подтверждением пароля, после чего нажать «Создать аккаунт» и подтвердить адрес электронной почты через письмо. Для входа в систему необходимо нажать кнопку «Войти» и ввести имя пользователя и

пароль. В случае утери пароля пользователь может воспользоваться функцией восстановления, указав email и следуя инструкциям из письма для установки нового пароля. Также предусмотрена возможность начать игру без регистрации, указав имя на главном экране.

Г.4.2 Создание и подключение к комнате

Для создания комнаты пользователь вводит имя и нажимает кнопку «Создать комнату», после чего открывается лобби с уникальным кодом, которым можно поделиться с другими игроками. Для подключения к существующей комнате необходимо ввести имя и код комнаты и нажать «Играть» либо перейти по прямой ссылке, полученной от другого участника.

Г.4.3 Настройка игры (для хоста)

Хост комнаты может настроить параметры игры в лобби, включая длительность хода с помощью таймера, уровень сложности слов и количество очков для победы. Он также может выбрать словарные коллекции из стандартных или пользовательских наборов. После подключения как минимум двух игроков хост запускает игру нажатием кнопки «Начать».

Г.4.4 Игровой процесс

После начала игры активному игроку отображается слово, которое он должен объяснить своей команде без использования самого слова, его производных или прямых переводов. Остальные участники вводят свои варианты ответов в поле ввода, и правильный ответ засчитывается автоматически. При необходимости слово можно пропустить с начислением штрафного очка. По истечении времени ход переходит к следующему игроку, а победителем становится тот, кто первым наберёт заданное количество очков.

Г.4.5 Голосовой чат

Во время игры отображается индикатор активности микрофона каждого участника в таблице очков. Пользователь может регулировать громкость других игроков через контекстное меню, вызываемое правой кнопкой мыши, а громкость собственного микрофона настраивается через системные параметры устройства.

Г.4.6 Пользовательские коллекции

Пользователь может создавать собственные коллекции слов через раздел «Мои коллекции», задавая название, описание и уровень сложности. Внутри коллекции можно добавлять слова, вводя их по одному на строку и сохраняя изменения. Также доступна функция удаления коллекции через соответствующую иконку.

Г.4.7 Профиль пользователя

В разделе «Профиль» пользователь может загрузить или удалить фотографию профиля, изменять имя пользователя и обновлять пароль, указывая текущий и новый пароль с последующим подтверждением изменений.

Г.5 Возможные ошибки и способы их устранения

Таблица 10 — Ошибки и способы их устранения

Ошибка	Причина	Способ устранения
«Комната не найдена»	Неверный код или комната удалена	Проверьте код комнаты, попросите хоста переслать ссылку

Продолжение Таблицы 10

«Игра уже началась»	Попытка войти в идущую игру новым игроком	Дождитесь следующей игры
«Соединение заблокировано»	Антивирус или брандмауэр блокирует WebSocket-трафик	Отключите сетевую защиту антивируса (Kaspersky, ESET), перезагрузите роутер или переключитесь на мобильный интернет / VPN
Письмо подтверждения не пришло	Письмо попало в папку «Спам»	Проверьте папку «Спам»; в разделе «Профиль» нажмите «Отправить письмо повторно»